The Surprising Effectiveness of Shared Memory in Looped Transformers

Báo cáo nghiên cứu: Kiến trúc LPT & H-LPT

Hung Manh

October 21, 2026

Nội dung trình bày

LPT: Tổng quan bài báo

- **Tên bài báo:** *The Surprising Effectiveness of Shared Memory in Looped Transformers*
- **Tác giả & Tổ chức:** Giovanni Monea, Keshav Ramji, Yousef El-Kurdi et al. (Cornell University & IBM Research, arXiv:2610.02383)
- **Ý tưởng cốt lõi:** Huấn luyện Looped Transformers chia sẻ bộ nhớ ngay từ đầu (pre-trained shared memory) thay vì lưu cache riêng cho từng bước lặp.
- **Đóng góp nổi bật:**
  - **Xóa bỏ định kiến:** Cắt giảm bộ nhớ không làm giảm chất lượng mà ngược lại giúp tăng mạnh độ chính xác và giảm Perplexity.
  - **Tiết kiệm bộ nhớ vượt bậc:** Giảm **76–79%** context memory so với Standard Transformer cùng kích thước ($150\text{M} \sim 1\text{B}$).
  - **Đại lộ Gradient (Gradient Highway):** Cơ chế đọc shared memory giúp dòng gradient từ hàm loss bước cuối truyền thẳng về vòng lặp đầu tiên, ổn định đệ quy sâu.

1. LPT: Bối cảnh & ”Cái bẫy” bộ nhớ KV Cache

Nguyên lý Looped Transformers (LT):

- Áp dụng cùng một khối L tầng Transformer lặp lại R lần trên hidden state trước khi dự đoán token:

$$h_{\text{in}}^{(t+1)} = \mathcal{N}_{\text{rec}}\left(h^{(t)} + \mathcal{M}^{L}(h_{\text{in}}^{(t)})\right)$$

- Đạt độ sâu tính toán hiệu dụng $L \cdot R$ nhưng chỉ tốn số lượng tham số của L tầng.

Cái bẫy bộ nhớ ngữ cảnh dài (Context Memory Explosion):

- Trong LT truyền thống, mỗi recursion $t \in [1, R]$ ghi một bảng KV cache riêng biệt.
- Tổng dung lượng cache: $\mathcal{O}(R \cdot L \cdot T \cdot d)$. Tại $T = 4096$, bộ nhớ KV cache phình to vượt xa bộ nhớ lưu trữ trọng số mô hình.
- *Thất bại của giải pháp cũ:* Bỏ bớt cache lúc suy luận (post-hoc inference drop như Ouro, Huginn) khiến chất lượng sinh sụp đổ nghiêm trọng.

2. LPT: Kiến trúc Looped Prediction Transformer

Phân tách Bộ nhớ và Dự đoán theo trục đệ quy:

- **State Recursion ($t = 1$):** Đọc toàn bộ ngữ cảnh, xây dựng và ghi lại bảng full-context KV cache duy nhất $\mathcal{C}^{(1)}$ trên toàn bộ L tầng.
- **Prediction Recursions ($t \geq 2$):** Không ghi thêm full cache, chỉ duy trì một cửa sổ trượt cục bộ siêu ngắn $\omega = 64$ token ($\mathcal{C}^{(t)}_{[i-\omega,i]}$, chiếm $< 2\%$ ngữ cảnh).

Attention đồng nhất qua một phép Softmax duy nhất:

$$\mathcal{A}_i^{(t)} = \mathcal{C}^{(1)}_{1:i} \cup \mathcal{C}^{(t)}_{[i-\omega,i]}$$

- Query $q_i^{(t)}$ chú ý đồng thời vào Shared Memory và cửa sổ cục bộ trong một lượt tính toán (tối ưu hóa qua FlexAttention block mask).
- **Biến thể Hybrid (H-LPT):** Kết hợp các tầng Linear Attention (Gated DeltaNet) với các tầng Full Attention dùng Shared Memory theo tỷ lệ 4:1.

3. LPT: Kết quả thực nghiệm phá vỡ biên giới Quality–Memory

Đánh giá trên FineWeb-Edu (Context $T = 4096$, Quy mô 150M – 1B tham số):

Mô hình	Recursions (R)	Val PPL (↓)	Zero-shot Acc
Standard Transformer (STD)	1	Baseline	Baseline
Vanilla LT (Full Cache)	5	Dừng cải thiện	Thấp hơn LPT 0.4
LPT (Ours)	5	**-1.09 nats** vs LT	**+4.8%** vs LT
H-LPT (Ours)	5	**-1.82 nats** vs STD	**+5.6%** vs ST

- **Quy luật mở rộng theo độ sâu:** LT truyền thống dừng cải thiện sau 3 vòng lặp; LPT tiếp tục tăng trưởng chất lượng mượt mà đến 5 vòng lặp.
- **Huấn luyện đơn giản:** LPT chỉ cần dùng Final Loss đơn giản ở bước cuối, không cần cơ chế Adaptive loss phức tạp như Ouro mà vẫn cực kỳ ổn định.

4. LPT: Giải mã cơ chế 1 – Đại lộ Gradient (Gradient Highway)

Nghịch lý suy giảm gradient trong đệ quy sâu:

- Trong LT chuẩn, gradient từ hàm loss ở bước cuối bị suy giảm nghiêm trọng khi truyền ngược qua nhiều bước lặp tuần tự $\rightarrow$ vòng 1 không nhận đủ tín hiệu.

Shared Memory đóng vai trò Gradient Highway:

- Do query ở các recursion sau đọc trực tiếp Shared KV cache của vòng 1, gradient có một đường dẫn tắt truyền thẳng về vòng 1.
- Chuẩn gradient truyền về vòng 1 trong LPT được phục hồi tương đương Standard Transformer, không bị sụt giảm theo số vòng lặp.

Thí nghiệm triệt tiêu (Ablation):

- Khi ngắt dòng gradient này (*detach shared KV*), mô hình ngay lập tức bị suy giảm chất lượng nặng nề hoặc không thể hội tụ khi tăng số vòng lặp.

5. LPT: Giải mã cơ chế 2 – Phân hóa biểu diễn & Vai trò của ω

Phân hóa không gian biểu diễn (PCA không gian Key):

- Phân tích PCA cho thấy Shared Keys và Local Keys tự động phân tách thành hai cụm không gian hoàn toàn riêng biệt.
- *Shared Keys:* Chuyên biệt hóa lưu trữ ngữ cảnh tĩnh toàn cục.
- *Local Keys:* Chuyên biệt hóa xử lý các tương tác cục bộ ngắn hạn.

Thí nghiệm quét kích thước cửa sổ ω ($0 \to 4096$):

- LPT vượt trội LT chuẩn ở **mọi kích thước cửa sổ** ω.
- Ngay cả khi $\omega = 0$ (hoàn toàn không lưu cache quá khứ ở các vòng sau), LPT vẫn tốt hơn LT giữ full cache.
- Tỷ trọng Attention: Càng về các recursion sau, mô hình càng dồn sự chú ý vào Shared Memory (chiếm tới **73–80%** attention mass).

6. LPT: Đo đạc thực tế trên phần cứng NVIDIA H100

Khắc phục nút thắt Memory-Bound khi phục vụ mô hình:

- **Khả năng chứa Batch (Batch Capacity):** Nhờ cắt giảm 79% context memory, H-LPT cho phép tăng batch size lên gấp **3.6× – 5.0×** so với Standard Transformer trước khi đầy 80GB VRAM.
- **Thông lượng thực tế (Throughput):**
  - Ở $R = 3$: H-LPT đạt thông lượng cao gấp **1.4 – 1.5×** so với Standard Transformer.
  - Ở $R = 5$: H-LPT duy trì thông lượng **81–88%** so với Standard Transformer, dù thực hiện số phép tính gấp nhiều lần.

Ý nghĩa thực tiễn

Việc tăng batch size trên GPU bù đắp hoàn toàn chi phí tính toán lặp lại, biến Looped Transformers thành giải pháp khả thi trong môi trường sản xuất thực tế.

7. LPT: Kết luận & Định hướng tương lai

- **Inductive Bias mới:** Chia sẻ bộ nhớ không phải là sự đánh đổi chấp nhận mất mát, mà là một cấu trúc kiến trúc vượt trội giúp phân định rõ ràng giữa lưu trữ ngữ cảnh và tinh chỉnh suy luận.
- **Đột phá đệ quy sâu:** Gradient Highway qua shared cache mở đường cho việc mở rộng Looped Transformers lên số vòng lặp lớn mà không gặp hiện tượng vanishing gradient.
- **Hiệu quả triển khai:** Giảm 76–79% bộ nhớ ngữ cảnh, tăng 5$\times$ batch capacity trên GPU hiện đại.

Hướng phát triển tiếp theo:

- Tích hợp cơ chế dừng sớm linh hoạt (adaptive compute / early exit) theo từng độ khó của token.
- Mở rộng sang các bài toán suy luận đa bước phức tạp (complex multi-step reasoning).

Cảm ơn bạn đã theo dõi!

Q&A